Logging in Python (Detailed Guide)

What is Logging?

Logging is the process of recording events, messages, warnings, errors, and debugging information while a Python program is running.

Instead of using `print()`, professional developers use **logging** because it provides better control over output and helps in debugging.

Why Use Logging?

Problems with `print()`

```
print("Program Started")
print("File Opened")
print("Something went wrong")
```

Problems:

- Cannot separate errors from normal messages.
- Difficult to disable later.
- Cannot save automatically to a file.
- Not suitable for large applications.

Advantages of Logging

- Record program execution
- Find bugs easily
- Save logs into files
- Different message levels
- Better than `print()`
- Used in Data Science, Web Development, AI, ML, Django, Flask, etc.

Import Logging

```
import logging
```

Basic Logging

```
import logging

logging.warning("This is a warning")
```

Output

```
WARNING:root:This is a warning
```

Logging Levels

Python has **5 main logging levels**.

Level	Value	Purpose
DEBUG	10	Detailed information for debugging
INFO	20	General program information
WARNING	30	Warning message
ERROR	40	Error occurred
CRITICAL	50	Serious error

---

1. DEBUG

Used while developing programs.

```
import logging

logging.basicConfig(level=logging.DEBUG)

logging.debug("Debug Message")
```

Output

```
DEBUG:root:Debug Message
```

---

2. INFO

General information.

```
logging.info("Program Started")
```

Output

```
INFO:root:Program Started
```

---

3. WARNING

Default logging level.

```
logging.warning("Low Disk Space")
```

Output

```
WARNING:root:Low Disk Space
```

---

4. ERROR

When an error occurs.

```
logging.error("Unable to Open File")
```

Output

```
ERROR:root:Unable to Open File
```

---

5. CRITICAL

Very serious error.

```
logging.critical("Database Crashed")
```

Output

```
CRITICAL:root:Database Crashed
```

Logging Hierarchy

```
CRITICAL
↑
ERROR
↑
WARNING
↑
INFO
↑
DEBUG
```

If the logging level is **WARNING**, then:

✓ WARNING

✓ ERROR

✓ CRITICAL

❌ INFO

❌ DEBUG

---

basicConfig()

Used to configure logging.

Syntax

```
logging.basicConfig(
    level=logging.INFO
)
```

Example

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info("Program Started")
logging.warning("Warning")
```

Logging to a File

```
import logging

logging.basicConfig(
    filename="app.log",
    level=logging.INFO
)

logging.info("Program Started")
logging.warning("Warning Message")
```

Output

A file named

```
app.log
```

contains

```
INFO:root:Program Started
WARNING:root:Warning Message
```

Overwrite or Append

Default (Append)

```
logging.basicConfig(
    filename="app.log"
)
```

Old logs remain.

Overwrite

```
logging.basicConfig(
    filename="app.log",
    filemode="w"
)
```

Old logs are deleted.

Formatting Logs

Default format

```
LEVEL:LOGGER:MESSAGE
```

Custom format

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s : %(message)s"
)

logging.info("Program Started")
```

Output

```
INFO : Program Started
```

Useful Format Specifiers

Specifier	Meaning
%(levelname)s	Log level

Specifier	Meaning
%(message)s	Message
%(asctime)s	Date & Time
%(filename)s	File name
%(funcName)s	Function name
%(lineno)d	Line number
%(name)s	Logger name

Example with Time

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.info("Program Started")
```

Output

```
2026-06-26 10:30:11 - INFO - Program Started
```

Logging Variables

```
import logging

name = "Krish"

logging.warning("Student Name: %s", name)
```

Output

```
WARNING:root:Student Name: Krish
```

Another example

```
age = 20

logging.info("Age = %d", age)
```

Using f-string (Works, but `%s` is preferred)

```
logging.info(f"Name = {name}")
```

Logging Exception

```
import logging

logging.basicConfig(level=logging.ERROR)

try:
    num = 10 / 0
except Exception as e:
    logging.error(e)
```

Output

```
ERROR:root:division by zero
```

Log Complete Exception Details

```
import logging

logging.basicConfig(level=logging.ERROR)

try:
    x = 10 / 0
except Exception:
    logging.exception("Exception Occurred")
```

Output

```
ERROR:root:Exception Occurred
Traceback (most recent call last):
...
ZeroDivisionError: division by zero
```

Logging Inside Functions

```
import logging

logging.basicConfig(level=logging.INFO)

def add(a, b):
    logging.info("Addition Started")
    return a + b

result = add(5, 3)

logging.info("Answer = %d", result)
```

Output

```
INFO:root:Addition Started
INFO:root:Answer = 8
```

Create Your Own Logger

```
import logging

logger = logging.getLogger("MyLogger")

logging.basicConfig(level=logging.INFO)

logger.info("Custom Logger")
```

Output

```
INFO:MyLogger:Custom Logger
```

Logging to File and Console Together

```
import logging

logger = logging.getLogger("App")
logger.setLevel(logging.INFO)

# File Handler
file_handler = logging.FileHandler("app.log")

# Console Handler
console_handler = logging.StreamHandler()

formatter = logging.Formatter(
    "%(asctime)s - %(levelname)s - %(message)s"
)

file_handler.setFormatter(formatter)
console_handler.setFormatter(formatter)

logger.addHandler(file_handler)
logger.addHandler(console_handler)

logger.info("Application Started")
```

This logs the message to both the console and the `app.log` file.

Real-Life Example

```
import logging

logging.basicConfig(
    filename="bank.log",
    level=logging.INFO,
```

```
    format="%(asctime)s - %(levelname)s - %(message)s"
)

balance = 5000

withdraw = 2000

if withdraw <= balance:
    balance -= withdraw
    logging.info("Withdraw Successful")
    logging.info("Remaining Balance = %d", balance)
else:
    logging.error("Insufficient Balance")
```

Logging Best Practices

- Use `DEBUG` for detailed debugging information.
- Use `INFO` for normal program events.
- Use `WARNING` for potential issues that don't stop the program.
- Use `ERROR` for recoverable errors.
- Use `CRITICAL` for severe failures that may stop the program.
- Configure logging (`basicConfig`) only once, usually at the start of the program.
- Prefer `logging.exception()` inside `except` blocks when you need the full traceback.
- Use placeholders like `%s` and `%d` instead of formatting strings yourself for better performance.

`print()` VS `logging`

Feature	`print()`	`logging`
Debugging	❌	✅
Log Levels	❌	✅
Save to File	❌	✅
Timestamp	❌	✅
Easy to Disable	❌	✅
Professional Use	❌	✅

Summary

- `logging` is used to record information about a program's execution.
- The five standard logging levels are `DEBUG`, `INFO`, `WARNING`, `ERROR`, and `CRITICAL`.
- Use `logging.basicConfig()` to configure logging behavior.

- Logs can be displayed on the console, written to a file, or both.
- `logging.exception()` is useful for recording exceptions with their full traceback.
- Logging is preferred over `print()` in production applications because it is more flexible, configurable, and easier to manage.

Link : CHATGPT

Rotating File Handler in Python (Detailed Guide)

What is Rotating File Handler?

A **Rotating File Handler** automatically creates a **new log file** when the current log file reaches a specified size.

This prevents one log file from becoming too large.

It is provided by the `logging.handlers` module.

Why Use Rotating File Handler?

Without rotation:

```
app.log
```

After several months:

```
app.log
(500 MB)
```

Problems:

- Very large file
- Slow to open
- Difficult to search
- Wastes disk space

With Rotating File Handler:

```
app.log
app.log.1
app.log.2
app.log.3
```

Old logs are automatically renamed, and a new `app.log` is created.

Import

```
import logging
from logging.handlers import RotatingFileHandler
```

Syntax

```
handler = RotatingFileHandler(
    filename,
    mode="a",
    maxBytes=0,
    backupCount=0
)
```

Parameters

Parameter	Description
`filename`	Name of the log file
`mode`	File mode (`a` = append, `w` = overwrite)
`maxBytes`	Maximum size of one log file (in bytes)
`backupCount`	Number of backup log files to keep

Example 1: Basic Rotating File Handler

```
import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger()

logger.setLevel(logging.INFO)

handler = RotatingFileHandler(
    "app.log",
    maxBytes=200,
    backupCount=3
)

logger.addHandler(handler)

for i in range(20):
    logger.info(f"Message {i}")
```

Output Files

```
app.log
app.log.1
app.log.2
app.log.3
```

When `app.log` reaches **200 bytes**, it is renamed:

```
app.log
↓
app.log.1
```

A new `app.log` is created automatically.

How Rotation Works

Suppose:

```
maxBytes = 100
backupCount = 3
```

Initially:

```
app.log
```

After reaching 100 bytes:

```
app.log.1
app.log
```

After the next rotation:

```
app.log.2
app.log.1
app.log
```

After the next rotation:

```
app.log.3
app.log.2
app.log.1
app.log
```

After another rotation:

```
app.log.3
app.log.2
app.log.1
app.log
```

The oldest backup (`app.log.3`) is deleted automatically.

Example 2: With Formatter

```
import logging
from logging.handlers import RotatingFileHandler
```

```
logger = logging.getLogger("MyLogger")

logger.setLevel(logging.INFO)

handler = RotatingFileHandler(
    "student.log",
    maxBytes=500,
    backupCount=2
)

formatter = logging.Formatter(
    "%(asctime)s - %(levelname)s - %(message)s"
)

handler.setFormatter(formatter)

logger.addHandler(handler)

for i in range(30):
    logger.info(f"Student Record {i}")
```

Sample Log

```
2026-06-26 11:20:10 - INFO - Student Record 1
2026-06-26 11:20:10 - INFO - Student Record 2
```

Example 3: Console + Rotating File

```
import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger("App")

logger.setLevel(logging.INFO)

file_handler = RotatingFileHandler(
    "app.log",
    maxBytes=500,
    backupCount=2
)

console_handler = logging.StreamHandler()

formatter = logging.Formatter(
    "%(levelname)s - %(message)s"
)

file_handler.setFormatter(formatter)
console_handler.setFormatter(formatter)

logger.addHandler(file_handler)
logger.addHandler(console_handler)

logger.info("Application Started")
```

Output on the console:

```
INFO - Application Started
```

The same message is also written to `app.log`.

Example 4: Error Logging

```
import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger()

logger.setLevel(logging.ERROR)

handler = RotatingFileHandler(
    "error.log",
    maxBytes=300,
    backupCount=2
)

logger.addHandler(handler)

try:
    x = 10 / 0
except Exception:
    logger.exception("An Error Occurred")
```

Sample `error.log`:

```
An Error Occurred
Traceback (most recent call last):
ZeroDivisionError: division by zero
```

Real-Life Example

```
import logging
from logging.handlers import RotatingFileHandler

logger = logging.getLogger("Bank")

logger.setLevel(logging.INFO)

handler = RotatingFileHandler(
    "bank.log",
    maxBytes=1024,
    backupCount=5
)

formatter = logging.Formatter(
    "%(asctime)s - %(levelname)s - %(message)s"
)
```

```
handler.setFormatter(formatter)

logger.addHandler(handler)

balance = 5000

withdraw = 2000

if withdraw <= balance:
    balance -= withdraw
    logger.info(f"Withdraw: {withdraw}")
    logger.info(f"Balance: {balance}")
else:
    logger.error("Insufficient Balance")
```

Difference Between FileHandler and RotatingFileHandler

Feature	FileHandler	RotatingFileHandler
Writes to file	✅	✅
File grows forever	✅	❌
Automatic rotation	❌	✅
Maximum file size	❌	✅
Backup files	❌	✅
Best for production	❌	✅

Important Parameters

`maxBytes`

Maximum size of the log file before rotation.

```
maxBytes = 1024
```

This means the file rotates after reaching **1024 bytes (1 KB)**.

`backupCount`

Number of old log files to keep.

```
backupCount = 5
```

Files created:

```
app.log
app.log.1
app.log.2
app.log.3
app.log.4
app.log.5
```

When another rotation happens, `app.log.5` is deleted, and the remaining files are shifted.

Advantages

- Prevents log files from becoming too large.
- Saves disk space by deleting the oldest backups.
- Automatically manages backup log files.
- Improves application performance and maintainability.
- Commonly used in production applications.

Best Practices

- Set an appropriate `maxBytes` based on expected log volume.
- Choose a reasonable `backupCount` to retain enough history without consuming excessive storage.
- Always use a formatter to include timestamps and log levels.
- Use `RotatingFileHandler` for applications that generate logs frequently.
- If you want logs to rotate based on **time** (daily, weekly, midnight), use `TimedRotatingFileHandler` instead.

Summary

- `RotatingFileHandler` rotates log files based on **file size**.
- It is imported from `logging.handlers`.
- `maxBytes` defines when rotation occurs.
- `backupCount` specifies how many backup files to retain.
- It is a better choice than `FileHandler` for long-running or production applications because it keeps log files organized and prevents unlimited growth.

Link : CHATGPT

Link : ALL